PROJECT HANDOVER

Expense Intelligence Platform

Context package for continuing in a new ChatGPT conversation

Owner	Status	Prepared
Terril Joel Nazareth	Scope complete; engineering design begins next	21 July 2026

Purpose: Give a new assistant enough context to continue as a technical reviewer without repeating the planning discussion or taking over implementation.

1. Executive handover

Terril is building a mobile-first, multi-user expense intelligence platform as a portfolio project aligned with Data, AI/ML, Cloud and DevOps engineering. The application converts receipts and financial records into auditable item-level spending data. It begins with reliable deterministic workflows and introduces OCR, retrieval and controlled AI only after the core data model works.

Current position: Project scope and target architecture are approved. No implementation review has occurred yet. The immediate next deliverable is the database schema and first vertical-slice technical design.

2. Working relationship

Terril will	The assistant will
Design and implement the application	Act as expert reviewer, evaluator and career-oriented technical coach
Create repository, code, infrastructure and documentation	Challenge architecture, scope, security, cost and unnecessary complexity
Share schemas, pull requests, errors or design choices	Give a clear verdict: ACCEPT, REVISE or REJECT, with reasons
Make final implementation decisions	Suggest next priorities and measurable acceptance criteria

Boundary: Do not build the application independently unless Terril explicitly asks for implementation. Prefer guidance, review, examples and targeted debugging.

3. Product definition

The intended user journey is:

Core flow: Upload receipt → extract structured data → validate and reconcile → human review → save through the application API → PostgreSQL and audit history → dashboard and drill-through.

The application should support:

- Mobile-friendly receipt capture and manual entry.
- Receipt-level and line-item-level records.
- Drill-through from month → category → merchant → transaction → receipt → product.
- Personal and shared expenses, including who paid and who benefited.
- Any number of household members through relational membership and allocation rows.
- Corrections, provenance, confidence and audit history.
- Graceful failure when OCR or LLM services are unavailable.
- Future retrieval, agentic enrichment and SQL-grounded questions without redesigning the core.

4. Users and multi-user model

Initial users are Terril and Joyline, but a third user must be addable without a schema migration. Never encode user-specific fields such as terril_share or joyline_share.

Entity	Purpose
users	Individual authenticated accounts

households	Shared expense spaces
household_members	Many-to-many membership with owner/admin/member roles
expense_allocations	One row per expense/user allocation using amount or percentage
invitations	Optional future invitation and acceptance workflow

5. Architecture baseline

Layer	Preferred choice	Responsibility
Client	React/Next.js PWA	Phone capture, review, dashboard and limited offline queue
API	FastAPI	Validation, authorization and application workflows
Database	PostgreSQL on Neon Free	Financial records, constraints, auditability and later pgvector
Object storage	Private Azure Blob Storage	Receipt images with short-lived access and lifecycle rules
Runtime	Azure Container Apps	Containerized API with scale-to-zero, min 0/max 1
OCR	Azure Document Intelligence F0	Primary receipt extraction; local PaddleOCR/Tesseract as fallback
LLM	GitHub Models	Structured fallback for unresolved, sanitized product text
LLM fallback	LLM.kiwi	Disposable secondary option; no personal or financial data
CI/CD	GitHub Actions + GHCR	Quality checks, immutable images, staging then controlled production release

6. Critical design rules

- The AI/LLM never writes directly to the database. It returns strict structured candidates; the application validates and saves.
- Money uses exact decimal/numeric types, never binary floating point.
- Every aggregate must link back to contributing source records.
- Preserve raw extraction, parsed value, corrected value, confidence, provider/model version and evidence separately.
- Duplicate detection and receipt-total reconciliation happen before completion.
- Every API request checks household membership and role authorization.
- External providers sit behind interfaces with timeouts, retries, quotas, feature flags and manual fallback.
- Use deterministic mappings before fuzzy search, embeddings, external APIs or an LLM.
- Do not log receipt content, bank data, tokens or personal data.

7. Scope priorities

Priority	Included capabilities
Must have	Manual receipt/transaction entry; item-level data; shared allocations; drill-through; auth; phone UI; review; tests; deployment
Should have	Bank CSV import; receipt matching; barcode lookup; offline capture; learned mappings; OCR; controlled retrieval
Could have	Natural-language SQL assistant; price comparison; forecasting; notifications; on-device preview
Not first release	Direct bank API; autonomous web research; native apps; Kubernetes; multiple autonomous agents; predictive ML on tiny data

8. Delivery path and milestones

Milestone	Outcome	Gate
M0 Engineering design	Repository structure, architecture, ERD, API contract, issues and Definition of Done	No unresolved core entity/workflow
M1 Manual vertical slice	Create receipt → add items → validate total → save → display	Traceable, authorized and tested
M2 Cloud/mobile baseline	Authenticated PWA and API deployed through CI/CD	Both users isolated; staging smoke tests pass
M3 Receipt intelligence	Upload, asynchronous extraction, review and reconciliation	8/10 clear receipts usable after review
M4 Learned normalization	Approved mappings, fuzzy retrieval and correction reuse	Repeated items resolve locally
M5 Controlled agent	Evidence-based unresolved-product candidates	No unsupported product auto-saved
M6 Analytics release	Auditable dashboards and optional SQL-grounded assistant	All totals reconcile and link to sources

9. DevOps baseline from the first commit

Use trunk-based development: short feature branch → pull request → main → automatic staging deployment → manual production approval. Every merged milestone must be deployed and observable.

Pull-request checks	Deployment controls
Ruff formatting/lint; mypy; Pytest; frontend lint/type check; migration validation; secret/dependency scan; Docker build; Trivy scan	Immutable image tagged with commit SHA; same image promoted to production; health/smoke checks; rollback; protected environment secrets

- Local development through Docker Compose.
- Infrastructure as code using Bicep where Azure resources are involved.
- Database migrations are versioned and validated.
- Expose /health/live and /health/ready.
- Structured logs include request/correlation ID, deployment version, duration, provider, status and estimated cost—never sensitive content.
- Dependabot, CodeQL/secret scanning, non-root containers, pinned dependencies and least-privilege identities.

10. Cost constraint

Hard limit: Normal target €0–€2 per month; absolute ceiling €5 per month. GitHub Student benefits are available, but the architecture must not depend on temporary credits.

Use	Avoid
Neon Free PostgreSQL; Container Apps scale-to-zero; Blob storage; Document Intelligence F0; GitHub Models free inference; local embeddings; Open Food Facts; GitHub Actions/GHCR	Azure PostgreSQL; Azure AI Search; VMs; Kubernetes; NAT Gateway; paid search APIs; paid AI subscriptions; always-on infrastructure

Controls: Azure alerts at €1/€3/€5; max one replica; paid GitHub Models disabled; cached mappings; compressed images; short log retention; provider quotas; OCR/LLM kill switches; no automatic free-tier upgrades.

11. Evaluation standard

Dimension	Weight	Evidence
Functional correctness	25%	Acceptance tests and reconciliation
Data quality	20%	Extraction and normalization metrics
Reliability	15%	Retries, duplicates and failure-path tests
Security/privacy	15%	Auth isolation, secrets and data handling
Cost efficiency	15%	Usage and cost report
User value	10%	Feedback from actual household users

Milestone release threshold: score ≥80%, no critical security failure, no unresolved money-calculation defect, projected monthly cost <€5, and a documented fallback or rollback.

12. Current status and next action

Completed	Not yet completed
Problem definition; resilient target architecture; multi-user approach; technology direction; 12-week milestone path; DevOps strategy; cost guardrails; project proposal	Repository setup; ERD/schema review; API contract; issue breakdown; implementation; deployment; evaluation dataset

Immediate task: Terril should draft the database schema for users, households, memberships, roles, receipts, receipt items, merchants, products/categories, expense allocations, product mappings, extraction attempts and audit events. The new assistant should review it before implementation.

13. Review checklist for the database schema

- Can a household add a third user without changing the schema?
- Are payer, beneficiary and allocation represented separately?
- Can receipt totals, item totals, discounts, deposits/Pfand, taxes and adjustments reconcile?
- Are currency and monetary precision explicit?
- Are original OCR values distinct from normalized and user-corrected values?
- Can duplicate receipts be detected without relying on one weak identifier?
- Are household boundaries enforceable through foreign keys and authorization?
- Can deletions/corrections be audited?
- Are timestamps, provenance, confidence and provider versions captured where needed?
- Does the schema support the first vertical slice without prematurely implementing RAG or agents?

14. Ready-to-paste prompt for the new chat

I am Terril Joel Nazareth. I am building the Expense Intelligence Platform described in the attached handover. I will implement the project myself. Act as my expert technical reviewer and evaluator—do not take over implementation unless I explicitly ask. Be direct and challenge weak assumptions, unnecessary complexity, security gaps, unreliable AI behavior and anything that risks the €5/month ceiling. For each artifact I share, give one verdict: ACCEPT, REVISE or REJECT; explain the important reasons; identify blocking issues; and state the next priority. Do not reopen approved scope unless new evidence requires it. We are currently at Milestone 0. My next artifact will be the database schema/ERD for the first vertical slice: manually create a receipt, add items, validate totals, save it and display it for an authorized household user. Review the schema for multi-user extensibility, financial correctness, auditability, privacy and future OCR/AI integration.

15. Useful files to attach in the new chat

Attach the handover first. Add the full project proposal if the new chat needs deeper milestone, risk or evaluation detail. Career documents are optional unless the discussion turns to CV positioning or job applications.

File	When useful
Expense_Intelligence_Platform_Project_Proposal.pdf / .docx	Detailed project plan and proposal
Terril_Joel_CV.pdf or Resume_2026-04.pdf	Portfolio wording, CV alignment or job targeting
Accenture Work Experience Letter.pdf; vwfs_reference.pdf	Evidence-based experience tailoring
Transcript of Records (English).pdf	Academic fit or course evidence
company_career_websites_germany.csv	Future job/thesis search—not needed for application design